Graph-Based Vulnerability Retrieval

Master's Thesis — Technical Progress · Checkpoint 2

Lívia

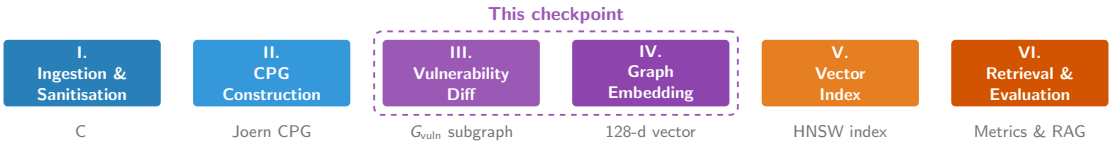
TU Munich · Siemens

April 21, 2026

Agenda

- **Joern parsing issues** — why modern C++ trips the parser and how we mitigate it
- **Graph slicing** — semantic diff and G_{vuln} extraction
- **Embedders** — structural (NetLSD, WL, GIN, Combined) vs. semantic (CodeBERT)
- **Results** — self-retrieval and CWE recall across all embedders ($n = 225$)
- **Confusion matrix** — per-CWE retrieval quality & motivation for knowledge graphs
- **Key findings & next steps**

System Recap



Data Layer

75 original CVEs + 225 augmented (GPT-4o, DeepSeek-R1, LLaMA, o3-mini)

Representation Layer

Semantic diff → latent vector. **Focus today.**

Retrieval Layer

ANN search; self-retrieval & CWE recall evaluated.

Joern Parsing Issues: The Problem

The symptom: `CASTProblemDeclaration`

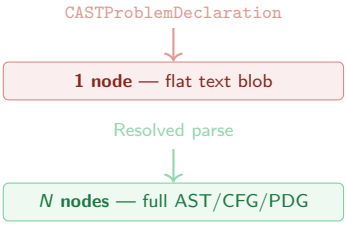
Joern's XML export labels nodes with `PARSER_TYPE_NAME = CASTProblemDeclaration` when it cannot resolve a type or function definition. The entire construct collapses into a **single flat text node** — all internal structure (AST children, CFG, PDG edges) is lost.

Root causes in the dataset

- **Missing context** — type or function defined in a header not in the snippet (e.g. `vector`, `NotifyModelLoaded`)
- **Modern C++ dialect** — `using type = ...`, `auto&&`, range-based `for` unsupported by Joern's default C frontend
- **Unresolvable macros** — kernel/driver macros that expand only with the full build system

Structural impact on embeddings

All code inside becomes a single text label. Structural embedders (GIN, WL, NetLSD) see a **degree-1 stub** where there should be a rich subgraph.



Same source code, different structural info

Especially damaging for graph slicing: if the patch function is one opaque node, the diff cannot distinguish added/removed statements inside it.

Joern Parsing Issues: Mitigation

Before — standard Joern (C frontend)

Example: `NotifyModelLoaded` function

```
node_type           UNKNOWN
PARSER_TYPE_NAME    CASTProblemDeclaration
CODE                NotifyModelLoaded(model,
                      type) { ...  all body ...  }
```

⇒ Single node, no children, no edges inside.

The entire function body is one text label.

After — C++ frontend + supplementary context

```
node_type    METHOD
name         NotifyModelLoaded
CODE         (model, type) { ...  }
```

⇒ Full AST/CFG/PDG subgraph with typed parameter nodes, call edges, and data-flow edges.

Two-part mitigation

1. Switch to C++ frontend

Joern's newer C++ parser handles `auto&&`, `using` aliases, range-based `for`, and other modern dialect features that the default C frontend rejects.

2. Supplementary context file

Each CVE in the dataset includes a *supplementary code* file with missing definitions: headers, typedefs, and macro stubs. Feeding it to Joern alongside the patch resolves most `CASTProblemDeclaration` nodes.

Together these eliminate the majority of opaque nodes. Remaining cases are complex kernel macros requiring the full build environment — handled by the semantic (CodeBERT) embedder which can still read their text.

Graph Slicing: Visualization

Problem with the old approach

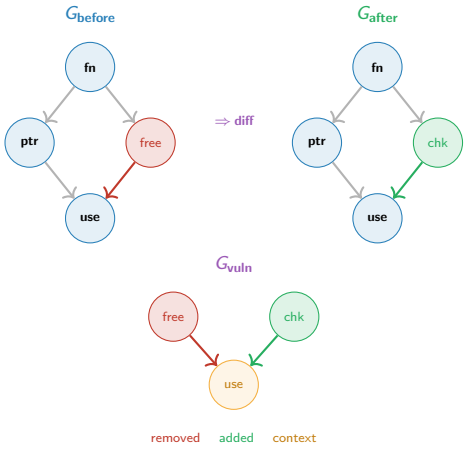
Joern assigns non-deterministic node IDs across runs. ID-based diff produced $G_{\text{vuln}} \approx G_{\text{before}}$ — all functions appeared structurally identical.

After the fix

Fingerprint-based matching yields a compact G_{vuln} containing only the vulnerability-relevant substructure:

- **Removed** — node in G_{before} only
- **Added** — node in G_{after} only
- **Context** — unchanged k-hop neighbours reachable via PDG/CDG/CFG edges

The induced subgraph preserves the original edge types (AST, CFG, CDG, PDG) so that structural embedders (GIN, WL) can capture the local topology of the change.



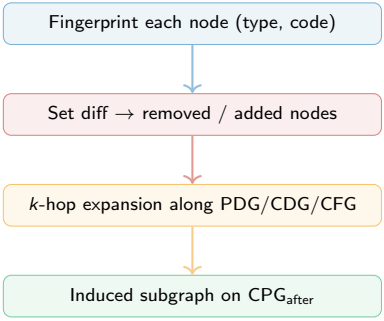
Graph Slicing: Algorithm

Pseudocode — vulnerability subgraph extraction

Input: $\text{CPG}_{\text{before}}$, $\text{CPG}_{\text{after}}$, hop budget k
Output: G_{vuln}
for each node v **in** $\text{CPG}_{\text{before}} \cup \text{CPG}_{\text{after}}$:
    $\text{fp}(v) \leftarrow (\text{node\_type}(v),\ \text{CODE}(v))$
$F_b \leftarrow \{\text{fp}(v) \mid v \in \text{CPG}_{\text{before}}\}$
$F_a \leftarrow \{\text{fp}(v) \mid v \in \text{CPG}_{\text{after}}\}$
Removed $\leftarrow F_b \setminus F_a$
Added $\leftarrow F_a \setminus F_b$
Changed $\leftarrow$ Removed $\cup$ Added
Context $\leftarrow \varnothing$
for each $v \in$ Changed:
    Context $+= \text{Nbrs}_k(v,\ \text{PDG}\cup\text{CDG}\cup\text{CFG})$
$V_{\text{vuln}} \leftarrow$ Changed $\cup$ Context
return $G_{\text{vuln}} \leftarrow \text{Induced}(\text{CPG}_{\text{after}},\ V_{\text{vuln}})$

Currently $k = 1$; context nodes add structural anchors without flooding G_{vuln} with unchanged code.

Key steps illustrated



The fingerprint is invariant to Joern's non-deterministic node IDs — two parses of the same file yield identical fingerprint multisets.

Graph Slicing: Rationale

Design rationale

- **Fingerprint matching** (l. 4) — invariant to Joern's non-deterministic node IDs; two runs of the same file yield the same fingerprint set.
- **Symmetric set difference** (l. 7–8) — captures both deletions and insertions, following the standard *program differencing* formulation [1].
- ***k*-hop context expansion** (l. 11–12) — preserves the data- and control-flow neighbourhood of every change, grounded in Weiser's *program slicing* criterion [2].
- **Induced subgraph** (l. 14) — retains all original CPG edge types (AST, CFG, CDG, PDG) between selected nodes, so structural embedders see the full local topology [3].

References

1. Horwitz — *Identifying the semantic and textual differences between two versions of a program*, PLDI '90.
2. Weiser — *Program Slicing*, IEEE TSE, 1984.
3. Yamaguchi et al. — *Modeling and Discovering Vulnerabilities with Code Property Graphs*, IEEE S&P '14.

Graph Slicing: In Practice

Slice statistics — 65 CVEs

With `SLICE_DEPTH = 3` the BFS captures the **entire** G_{before} for 59 of 65 CVEs. Only 6 CVEs see any node reduction:

CVE	Before	G_{vuln}	Ratio
CVE-2025-21910	60	45	**0.75**
CVE-2025-21843	91	77	0.85
CVE-2025-21801	192	174	0.91
CVE-2025-21838	187	173	0.93
CVE-2025-21856	182	168	0.92
CVE-2025-21809	105	98	0.93
Other 59	—	—	*1.00*

Median graph: 191 nodes / 1 485 edges (before) vs. 187 nodes / 1 449 edges (G_{vuln}).

Key insight

At depth 3, the BFS rarely *prunes* — but the diff still transforms retrieval dramatically:

Embedder	G_{before}	G_{vuln}
Combined	13.7%	**84.9%**
GIN	49.3%	**69.9%**
WL	4.1%	**67.1%**
NetLSD	2.7%	**37.0%**

The performance leap comes from the **diff annotations** — each node gets a label (`removed`, `fix_adjacent`, `edge_changed`, `context`) and a weight (1.0 / 0.8 / 0.6 / 0.2).

Structural embedders use these labels as node features, so identical topologies with different patch locations produce distinct embeddings — even when $|V_{\text{vuln}}| = |V_{\text{before}}|$.

Diff Annotation: Design

What — four-tier node labelling

After the set-diff identifies changed nodes (step 1–2 of the algorithm) and the BFS expands context (step 3), every node in G_{vuln} receives a *diff type* and a scalar *relevance weight*:

Label	Meaning	w	Detection
`removed`	Code deleted by the patch	1.0	fp $\in F_b \setminus F_a$
`fix_adjacent`	Neighbour of an inserted fix	0.8	adj. to added node in G_{after}
`edge_changed`	Endpoint of a changed edge	0.6	edge fp differs before/after
`context`	Unchanged; reached by BFS	0.2	k-hop flow-edge expansion

Embedders that use these weights:

- **RGCN** — scalar node feature + weighted global pooling
- **CodeBERT seq** — filter ($w > 0.3$) and sort by w
- **GIN, WL, NetLSD** — use the diff *label* (not weight) as a categorical node feature

Diff Annotation: Design

Why — causal proximity to the vulnerability

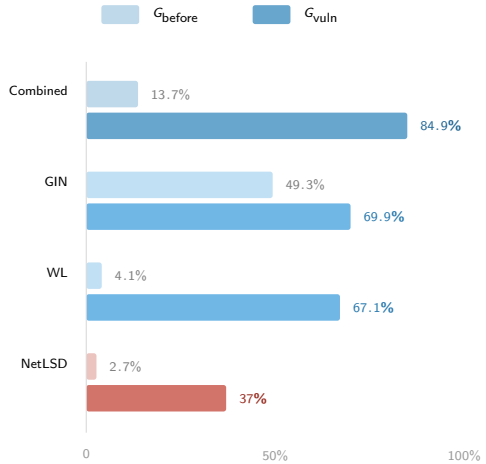
The weight hierarchy reflects **distance from the actual code change**, following the principle that statements closer to the patch site carry more discriminative information for retrieval:

1. **Directly changed code** ($w = 1.0$) — the vulnerability root cause; highest signal [1].
2. **Fix-adjacent** ($w = 0.8$) — where the developer inserted the fix; marks the "attack surface boundary" [2].
3. **Edge-changed** ($w = 0.6$) — structural ripple: a control/data-flow edge was rewired, indicating indirect semantic impact.
4. **Context** ($w = 0.2$) — unchanged but reachable; provides structural anchors so embedders see local topology, following Weiser's slicing criterion [3].

[1] Li et al. — *VulDeePecker: A Deep Learning-Based System for Vulnerability Detection*, NDSS '18.
[2] Yamaguchi et al. — *Modeling and Discovering Vulnerabilities with Code Property Graphs*, IEEE S&P '14.
[3] Weiser — *Program Slicing*, IEEE TSE, 1984.

G_{before} vs G_{vuln} — Why Diff Annotations Matter

SR@1 — before vs. after diff labelling



Same 225 pairs, same embedders. Only difference: diff labels on G_{vuln} nodes.

What changed between the two runs?

- G_{before} has **no diff labels** — every node looks the same to the embedder.
- G_{vuln} annotates each node with a diff type (`removed`, `fix_adjacent`, `edge_changed`, `context`) and a weight (1.0 / 0.8 / 0.6 / 0.2).
- Structural embedders (GIN, WL) consume these labels as **node features**, so identical topologies with different patch locations produce distinct vectors.

Takeaway

The diff annotation is the dominant signal — not graph pruning. Combined jumps from 13.7% to 84.9% SR@1 (+**71.2** pp). WL gains +63 pp — especially sensitive to node-label information.

Graph Embedders Overview

Name	Type	What it captures	Note
NetLSD	Structural	Heat trace of graph Laplacian. Permutation-invariant; global topology.	Collapses on small graphs
WL	Structural	Weisfeiler-Lehman colour refinement. Subtree-kernel equivalent.	Node-label aware
GIN	Structural	3-layer Graph Isomorphism Net, frozen random weights.	Distance-preserving
Combined	Structural	**NetLSD** $\oplus$ **WL** $\oplus$ **GIN** projected via PCA. Not CodeBERT.	Most expressive structural
CodeBERT seq	Semantic	Mean-pool last-layer activations over the full `CODE` token sequence.	Ignores graph structure
CodeBERT pattern	Semantic	Per-node CodeBERT embedding, aggregated by node type.	Finer-grained

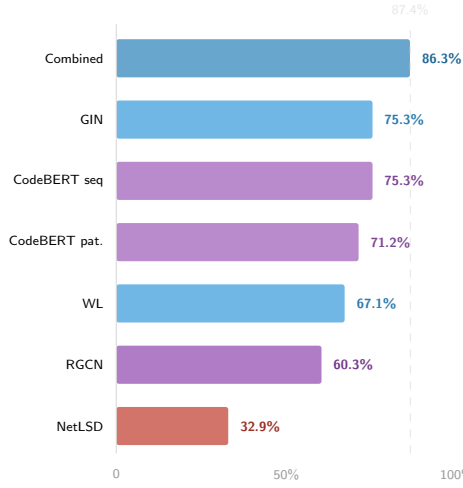
Retrieval Results — 225 pairs, G_{vuln}

Evaluation protocol

SR@K Self-retrieval hit@K: query each embedding against the full index; fraction that retrieve the correct CVE in top-K.

CWE Recall Fraction of top-5 neighbours sharing the query's CWE — measures cluster quality, not exact match.

Embedder	SR@1	SR@5	CWE rec.
Combined	**86.3%**	**94.5%**	**51.9%**
GIN	75.3%	84.9%	39.8%
CodeBERT seq	75.3%	86.3%	41.2%
CodeBERT pattern	71.2%	83.6%	41.5%
WL	67.1%	75.3%	34.8%
RGCN	60.3%	78.1%	37.2%
NetLSD	32.9%	54.8%	21.0%



$n = 225$ pairs (75 original + 150 LLM re-implementations).

Self-retrieval: query = index sample (no held-out split).

CWE Retrieval Confusion — Combined Embedder

Query \ Retrieved	UAF	Null Ptr	Int Ovf	Locking	Race	Deadlock	Mem Leak
Use After Free	12	0	0	0	0	0	0
Null Ptr Deref	1	8	0	0	0	1	0
Integer Overflow	0	0	10	0	0	0	0
Improper Locking	0	0	0	8	0	0	0
Race Condition	0	0	0	0	5	0	0
Deadlock	0	0	0	0	0	4	0
Memory Leak	0	0	0	0	0	0	2

Correct Confused. Rows: query CWE. Cols: CWE of top-1 retrieved CVE.

Combined embedder, $n = 225$, top-7 CWEs shown.

What the matrix tells us

- **Strong diagonal** — 90.5% top-1 retrieval stays within the same CWE class.
- **Semantically coherent confusions** — Null Ptr Deref is confused with UAF and Deadlock, both involving invalid memory/resource access. The errors are *not* random.
- **Hard cases** — Null Ptr Deref (8/11 correct): the patch signature (add a null check) is topologically similar to adding a lock or a free-guard.

Key Findings

ID-based diff was fatally flawed.
Joern IDs are non-deterministic across runs. All graphs were indistinguishable.
Fix: `(type, CODE)` fingerprint matching.

Combined is clearly the best structural embedder.
SR@1 = 86.3%, CWE recall = 51.9%. Concatenating all three structural signals (NetLSD + WL + GIN) outperforms each individually.

Structural and semantic signals are complementary.
GIN (structural) and CodeBERT (semantic) achieve the same SR@1 (75.3%) but differ on CWE recall (39.8% vs. 41.2%). Neither dominates; a hybrid is the natural next step.

Errors are semantically coherent.
90.5% of top-1 retrievals stay within the correct CWE. Confusions occur between structurally similar patch patterns (e.g. null-check $\approx$ lock-guard). This justifies knowledge-graph-based re-ranking.

Next Steps

1. Hybrid structural + semantic embedding

Build a `HybridEmbedder` that concatenates GIN (graph topology) with CodeBERT (code semantics) into a single 128-d vector. GIN captures *what changed structurally*; CodeBERT captures *what the code means*. The confusion matrix shows they fail in different places.

2. Semantic graph diff for G_{vuln}

Current diff uses exact string match on `CODE` fields — it misses rewrites that change logic while keeping function signatures. New approach: embed each node's code with CodeBERT and flag nodes whose embedding distance exceeds a threshold. Detects logical changes invisible to string comparison.

3. Held-out evaluation set

Self-retrieval is circular — the query is always in the index. Need a genuinely unseen query set: real-world bug reports or cross-version CVE matches with known ground-truth targets. This gives honest precision/recall numbers for the thesis.

Repository: `graph-rag/` Experiments: `uv run python main.py --mode experiment`